Experiment 9 — Alumnex Connect Conceptual Classes and UML Domain Model

Description

Experiment 9 — Alumnex Connect Conceptual Classes and UML Domain Model

Modeling basis

The class model is based on the actual Mongoose models under `backend/models/` and API/service modules in `backend/routes/` and `backend/services/`. The frontend's existing React pages form the Boundary/UI layer; Express route modules and service modules implement the Control layer; Mongoose models form the persistent Entity layer. The model intentionally uses existing source concepts instead of inventing a Course/Examination/Marksheet subsystem that is not the product's core domain.

Repository: FSD/alumni-connect-website-main at master · SrimanKumarV/FSD
Key sources: User.js, Mentorship.js, Job.js, ActivityGoal.js, Message.js, Event.js.

Boundary classes — existing UI pages

Details

Assignee
subhadharanis.24cse
Assign to me

Parent
ACS-28 Alumnex Connect — Scrum Lab Ex

Sprint
Add sprint

Labels
experiment-9 scrum-lab

Due date
Add due date

Team
Add team

`LoginPage`, `RegisterPage`, `ProfilePage`, `MentorshipPage`, `JobsPage`, `ResumeAnalyzerPage`, `DevPulsePage`, `ActivityHubPage`, `ChatPage`, `NotificationsPage`, `EventsPage`, `CalendarPage`, `ForumPage`, `ProjectShowcasePage`, `ContestsPage`, `AdminPage`.

These are conceptual Boundary labels for existing React pages/routes; they do not assert that each is an independently named class in the source.

Control classes/modules — existing API and services

- Authentication and authorization: `backend/routes/auth.js`, `backend/routes/users.js`, `backend/middleware/auth.js`, `backend/services/authService.js`.
- Mentorship: `backend/routes/mentorship.js`, `backend/services/mentorshipService.js`.
- Career: `backend/routes/jobs.js`, `backend/services/jobService.js`, `backend/routes/ai.js`.
- Activity: `backend/routes/activity.js`, `backend/routes/devActivity.js`, `backend/services/activityService.js`.
- Messaging and notification: `backend/routes/messages.js`, `backend/routes/notifications.js`, Socket.IO handler, push/notification services.
- Event/calendar: `backend/routes/events.js`, `backend/routes/googleCalendar.js`, `backend/services/googleCalendarService.js`, `calendarSyncService.js`.
- Community: `backend/routes/forum.js`, `tech-hub.js`, `projects.js`, `contests.js`, `institutions.js`, `business.js`.
- Governance: `backend/routes/admin.js`.

Start date

Add date

Story point estimate

Add story points

Reporter

SV SRIMAN VEL

Development

Connect development tools

Open in coding tool

Create branch

Create commit

Automation

Recent rule runs Refresh

Refresh to see recent runs.

Create new automation rule

Created 1 day ago

Updated 30 minutes ago

Configure

These are existing modules used as the Control layer, not claimed separate `*Controller` classes.

Entity classes

`User`, `MentorshipRequest`, `Mentorship`, `MentorshipSession`, `MentorReview`, `MentorReward`, `Job`, `JobApplication`, `DevProfile`, `ActivityGoal`, `ActivityRecord`, `Message`, `ChatGroup`, `Notification`, `NotificationDevice`, `ReminderPreference`, `Event`, `GoogleCalendarConnection`, `GoogleCalendarEvent`, `ForumPost`, `Project`, `Contest`, `Business`, `TechHub` question/reply entities and `AdminIncident`.

Key relationships and multiplicities (conceptual, align to model references)

- A User can have one or more mentorship requests as student and/or mentor.
- A Mentorship links the student and mentor and can be accompanied by sessions, milestones/notes, reviews and reward totals.
- A Job can have many JobApplication records; each application belongs to one applicant and one job.
- A User owns developer/activity/profile settings and activity records/goals.
- A Message has a sender and recipient and may belong to a conversation/group; notification records belong to recipient and may point to a triggering object.
- A User may have notification device registrations, reminder preferences and a Google Calendar connection; GoogleCalendarEvent records associate a member/calendar/event identifier.
- User-authored ForumPost and Project records are owned by an author; contests have participation/submission/leaderboard workflows.

Mermaid UML class diagram source

```
classDiagram
  class User {
    +ObjectId id
    +string name
    +string email
    +string role
    +string college
    +string department
    +boolean isVerified
    +boolean isApproved
    +boolean isActive
  }
  class MentorshipRequest {
    +ObjectId student
    +ObjectId mentor
    +string domain
    +string status
  }
  class Mentorship {
    +ObjectId student
    +ObjectId mentor
    +string status
    +date startDate
    +date endDate
    +boolean completed
  }
  class MentorshipSession {
    +ObjectId student
    +ObjectId mentor
    +date date
    +string time
    +string status
  }
  class MentorReview {
    +ObjectId mentorId
    +ObjectId studentId
```

```
    +number rating
  }
  class Job {
    +string title
    +string company
    +string status
    +string applicationLink
  }
  class JobApplication {
    +ObjectId job
    +ObjectId applicant
    +string status
  }
  class DevProfile {
    +ObjectId user
    +string platform
    +string username
  }
  class ActivityGoal {
    +ObjectId userId
    +string title
    +boolean isActive
  }
  class ActivityRecord {
    +ObjectId userId
    +string source
    +date activityDate
  }
  class Message {
    +ObjectId sender
    +ObjectId receiver
    +string content
    +string status
  }
  class ChatGroup {
    +string name
    +string type
  }
  class Notification {
```

```
    +ObjectId recipient
    +ObjectId sender
    +string type
    +boolean isRead
  }
  class NotificationDevice {
    +ObjectId userId
    +string platform
    +boolean enabled
  }
  class Event {
    +string title
    +date startDate
    +date endDate
    +string status
  }
  class GoogleCalendarConnection {
    +ObjectId userId
    +boolean connected
  }
  class GoogleCalendarEvent {
    +ObjectId userId
    +string googleEventId
    +string calendarId
  }
  class ForumPost {
    +ObjectId author
    +string title
    +string status
  }
  class Project {
    +ObjectId user
    +string title
    +string githubLink
    +string liveLink
  }
  class Contest {
    +string title
    +string status
```

```
  }
  User "1" --> "0..*" MentorshipRequest : student/mentor
  User "1" --> "0..*" Mentorship : student/mentor
  Mentorship "1" --> "0..*" MentorshipSession : sessions
  Mentorship "1" --> "0..*" MentorReview : feedback
  User "1" --> "0..*" JobApplication : applies
  Job "1" --> "0..*" JobApplication : receives
  User "1" --> "0..*" DevProfile : owns
  User "1" --> "0..*" ActivityGoal : sets
  User "1" --> "0..*" ActivityRecord : generates
  User "1" --> "0..*" Message : sends/receives
  ChatGroup "1" --> "0..*" Message : contains
  User "1" --> "0..*" Notification : receives
  User "1" --> "0..*" NotificationDevice : registers
  User "1" --> "0..1" GoogleCalendarConnection : connects
  User "1" --> "0..*" GoogleCalendarEvent : syncs
  User "1" --> "0..*" ForumPost : authors
  User "1" --> "0..*" Project : publishes
```

Modeling caveat

This is a conceptual UML view, not an exact generated diagram of all Mongoose validators/indexes. Check schema fields and cardinality constraints in source before using it as a database migration specification. The current updated artifact ZIP now contains Alumnex-specific diagram PNG/SVG/Mermaid sources. Attachments in Jira are tracked separately by ACS-46.

Rendered diagram evidence (updated for this repository study)

The current Alumnex-specific domain diagram is in the latest `ACS_Scrum_Experiments_Artifacts.zip`:

- `diagrams/domain_model.png`

- `diagrams/domain_model.svg`
- `diagrams/domain_model.mmd`
- `diagrams/class_roles.md`

The former generic student/course/examination image was replaced in the package with an Alumnex-specific model of actual Mongoose entities. The image files are **not attached to this Jira issue yet** because Atlassian Media upload failed from the execution environment. Manual attachment follow-up: ACS-46.

Attachments

Add attachment | Create project plan Try

Subtasks

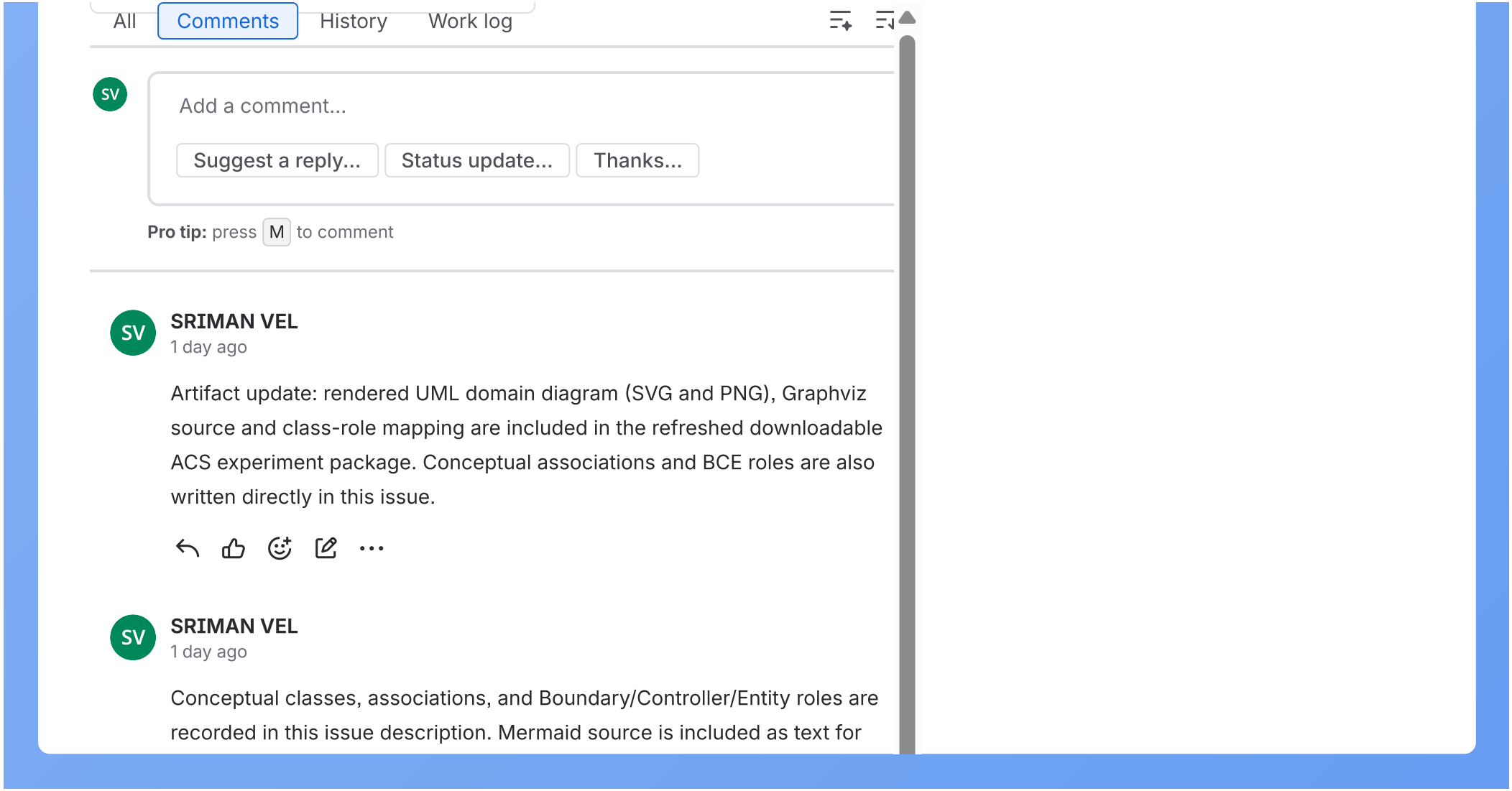
0% Done

Work	P...	A...	Statu
ACS-46 Experiments 8–9 — Attach UML Diagram Exports to Jira			To D

Linked work items

Add linked work item

Activity


All
Comments
History
Work log
SV
Add a comment...
Suggest a reply...
Status update...
Thanks...
Pro tip: press M to comment
SV
SRIMAN VEL
1 day ago
Artifact update: rendered UML domain diagram (SVG and PNG), Graphviz source and class-role mapping are included in the refreshed downloadable ACS experiment package. Conceptual associations and BCE roles are also written directly in this issue.
SV
SRIMAN VEL
1 day ago
Conceptual classes, associations, and Boundary/Controller/Entity roles are recorded in this issue description. Mermaid source is included as text for